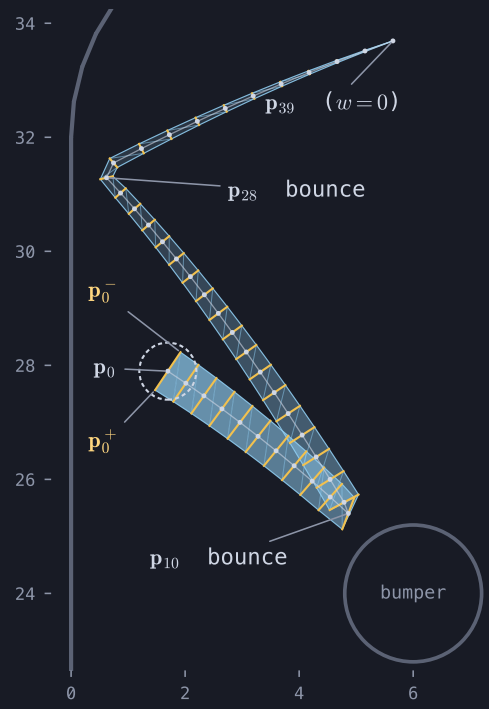
Building the ribbon

To turn the saved positions into a ribbon, each one gets a vertex on either side of the path. The side direction is perpendicular to the way the ball was moving there, which we get from the two neighboring points:

$$\mathbf{p}_i^{\pm} = \mathbf{p}_i \pm w_i\,\hat{\mathbf{n}}_i, \qquad w_i = 0.4\,(1 - a_i), \qquad a_i = \frac{i}{39}$$

Here $\mathbf{p}_i$ is a saved position with $i = 0$ the newest, and $\hat{\mathbf{n}}_i$ is the unit vector perpendicular to the path. a_i is the age of the point, from 0 at the ball to 1 at the tail, so the half-width w_i shrinks from 0.4 to 0 and the ribbon tapers to a point.



The same 40 points in table space. Gold rungs connect each $\mathbf{p}_i^+$ to $\mathbf{p}_i^-$, and the dashed circle is the ball. Where the ball reverses at the bumper, the ribbon twists through a point.

The perpendicular is the direction of travel, $\mathbf{p}_{i-1} - \mathbf{p}_{i+1}$, rotated $90°$ by swapping its components and negating one, then divided by its length. The ends of the array reuse their one neighbor. The vertices go into a buffer in the order $\mathbf{p}_0^+, \mathbf{p}_0^-, \mathbf{p}_1^+, \mathbf{p}_1^-, \ldots$, which is what `gl.TRIANGLE_STRIP` wants, and every pair is lifted a little above the playfield so the ribbon doesn't z-fight with it.

```
trail.forEach(([x, y], i) => {
  const [ax, ay] = trail[Math.max(i - 1, 0)];
  const [bx, by] = trail[Math.min(i + 1, trail.length - 1)];
  const len = Math.hypot(ax - bx, ay - by) || 1;
  const age = i / (TRAIL_LENGTH - 1);
  const w = TRAIL_WIDTH * (1 - age);
  const nx = -(ay - by) / len * w;
  const ny = (ax - bx) / len * w;
  trailPositions.set([x + nx, y + ny, TRAIL_HEIGHT,
                      x - nx, y - ny, TRAIL_HEIGHT], i * 6);
  trailTexCoords.set([age, 1, age, -1], i * 4);
});
```

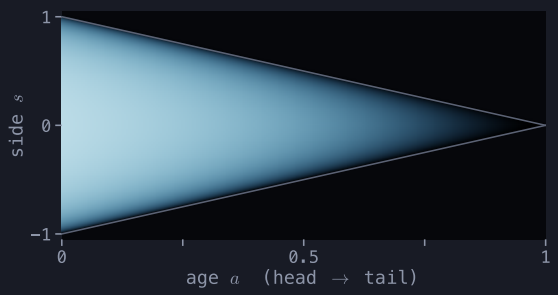
Fading and glow

Each vertex also gets a texture coordinate: `x` is its age a, and `y` is its side s, which runs from -1 at one edge through 0 in the middle to $+1$ at the other edge. The fragment shader turns those into a brightness:

$$\mathbf{C}(a, s) = \mathbf{C}_{\text{trail}}\,(1 - s^2)\,(1 - a)^2$$

```
float fade = 1.0 - vTexCoord.x;
float core = 1.0 - vTexCoord.y * vTexCoord.y;
fragColor = vec4(uColor * core * fade * fade, 1.0);
```

So the streak is brightest down the middle and at the ball, dark at the edges, and fades toward the tail. Halfway down the trail the middle is a quarter as bright as the head.



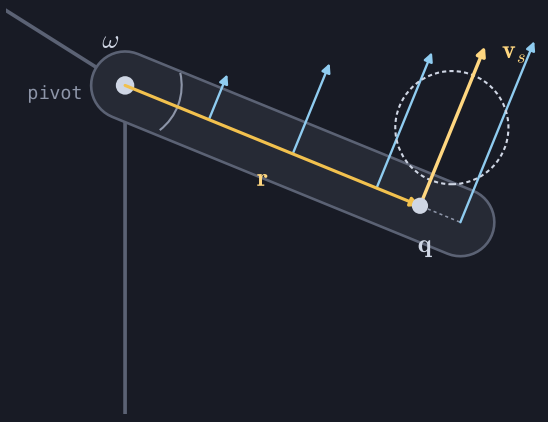
The ribbon unrolled: brightest at the head and along the middle.

How fast the flipper is moving

Every substep the ball is tested against the flipper's centerline, the segment from its pivot to its tip. **q** is the point on that segment closest to the ball, and **r** points from the pivot to **q**. Because the flipper turns about its pivot, **q** moves perpendicular to the flipper:

$$\mathbf{v}_s = \omega\,(-r_y,\ r_x), \qquad \|\mathbf{v}_s\| = |\omega|\,\|\mathbf{r}\|$$

ω is the flipper's angular velocity. The speed grows with distance from the pivot, so contact near the tip can give a stronger hit. A rising flipper turns at 18 rad/s, so its tip, 3.2 units out, moves at almost 58 units/s.



Blue arrows show how fast each part of the flipper moves; $\mathbf{v}_s$ is the velocity at the contact point.

In code, `collideFlippers()` works out **q**, **r** and $\mathbf{v}_s$ for each flipper and hands them to `resolve()`, along with the contact distance (ball radius plus flipper radius) and the flipper's restitution of 0.3:

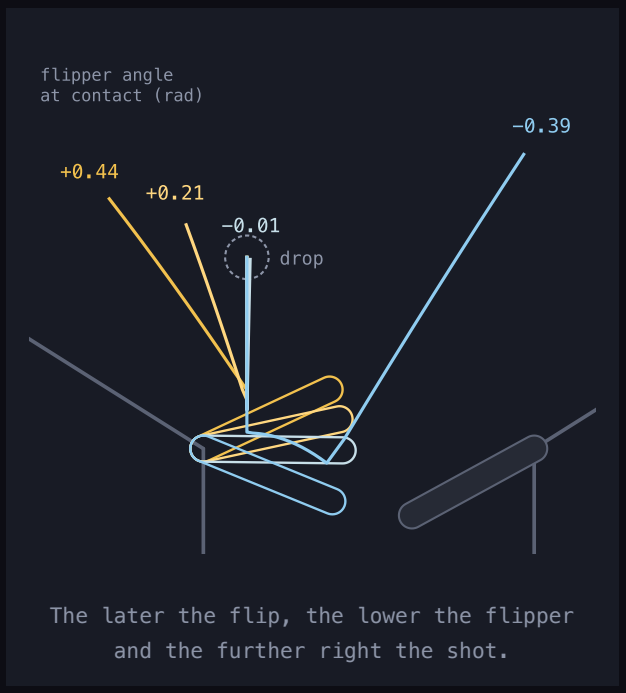
```
function collideFlippers(ball) {
  for (const f of table.flippers) {
    const q = closestOnSegment(ball.p, f.pivot, flipperTip(f));
    const r = sub(q, f.pivot);
    const vs = [-f.omega * r[1], f.omega * r[0]];
    resolve(ball, q, BALL_RADIUS + f.radius, 0.3, vs);
  }
}
```

The bounce

`resolve()` takes **n**, the unit vector from **q** toward the ball, and pushes the ball out of the flipper along it. If the ball and the flipper surface are moving toward each other along **n**, it reverses that part of the ball's velocity, measured relative to the moving surface:

$$\mathbf{v}_{\text{out}} = \mathbf{v}_{\text{in}} - 1.3\left[(\mathbf{v}_{\text{in}} - \mathbf{v}_s)\cdot\mathbf{n}\right]\mathbf{n}$$

This only happens when the dot product is negative. The 1.3 is $1+e$, with the flipper's restitution $e = 0.3$.



The later the flip, the lower the flipper and the further right the shot.

```
function resolve(ball, q, reach, restitution, vs = [0, 0]) {
  const d = sub(ball.p, q);
  const dist = Math.hypot(d[0], d[1]);
  if (dist >= reach || dist === 0) return false;
  const n = [d[0] / dist, d[1] / dist];
  ball.p[0] += n[0] * (reach - dist);
  ball.p[1] += n[1] * (reach - dist);
  const vn = dot(sub(ball.v, vs), n);
  if (vn < 0) {
    ball.v[0] -= (1 + restitution) * vn * n[0];
    ball.v[1] -= (1 + restitution) * vn * n[1];
  }
  return true;
}
```

So the shot comes from the moment of contact. Pressing earlier or later changes the flipper's angle when it meets the ball, which changes **n** and the direction the ball leaves in.

The flipper's material has a dark ordinary color and uses the palette color as its emissive term, which adds brightness directly, even where the lights leave the surface dark. `draw()` scales it by the brightness and sends it to the shader:

```
flipper: { color: [0.1, 0.1, 0.1], ks: 0.4, shininess: 40, emissive: palette.flipper },

draw(flipperMesh, multiply(frame, translate(0, 0, 0.25), scale(1, 1, h - 0.25)),
     MATERIALS.flipper, f.pressed ? 1.6 : 1);

gl.uniform3fv(uniforms.uEmissive, (mat.emissive ?? BLACK).map(v => v * boost));
```

The fragment shader adds the emission to the ordinary lighting. For the flipper `uTopEmissive` is zero, so this is just `result += emission`, and the flipper comes out bright but sharp-edged:

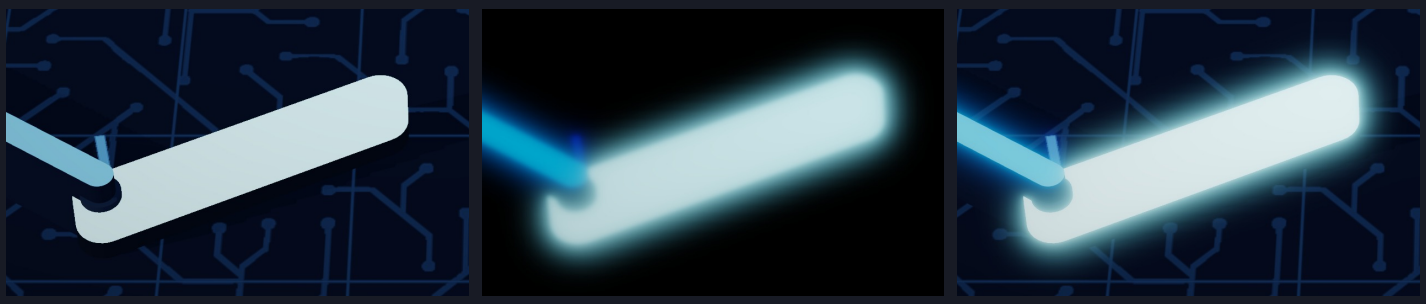
```
result += emission + uTopEmissive * smoothstep(0.7, 0.9, dot(N, uUp));
```

Bloom

The bloom pass keeps only the brightness above a threshold of 1 and blurs it horizontally and vertically. It repeats that over four smaller and smaller images, which gives both a narrow and a wide glow. The final shader adds the blur back to the scene:

$$\mathbf{H} = 1.4\,(\mathbf{S} + 0.35\,\mathbf{B})$$

S is the original scene and **B** is the combined blur, before the final display conversion.



Left: the scene. Middle: the blurred bright parts. Right: the two combined.

```
return max(texture(uImage, uv).rgb - uThreshold, 0.0);

vec3 hdr = EXPOSURE * (texture(uScene, vUv).rgb + BLOOM_STRENGTH * bloom);
```

Pressing a flipper raises its emission, which gives the bloom more brightness to spread around its edges.